МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
Кафедра комп'ютерних інформаційних технологій

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Якість програмного забезпечення та тестування»

на тему: «Тестування методами білої скриньки»

Виконала: студентка
групи ПЗ2313
Бондаренко Поліна Євгенівна

Перевірила:
к.т.н., доцент кафедри КІТ
Куроп’ятник Олена Сергіївна

Дніпро 2026

1. ТЕМА ТА МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Тема: Тестування методами білої скриньки.

Мета роботи: Отримати практичні навички проектування та виконання тестів методами «білої скриньки» (White-Box Testing).

2. ПОСТАНОВКА ЗАДАЧІ (ЗМІСТ РОБОТИ)

Відповідно до навчальної програми дисципліни та методичних вказівок, зміст роботи включає:

1. Ознайомитися з теоретичними засадами методів тестування програмного забезпечення за принципом білої скриньки.

2. Виявити переваги та недоліки методів тестування білою скринькою.

3. Підготувати дві тестовані функції мовою TypeScript для предметної області «Автоматизація кав'ярні», що задовольняють вимогам:

- містять щонайменше чотири керуючі конструкції (розгалуження `if`, цикли `for`);
- містять складені логічні умови зі зв'язками кон'юнкції (`&&`) та диз'юнкції (`||`) для повноцінної демонстрації відмінностей між методами покриття;
- повертають значення через оператор `return`, не використовують генератор випадкових чисел та виведення в консоль як єдиний результат;
- забезпечені повною специфікацією (призначення, типи та діапазони параметрів, точні правила обчислення).

4. Розробити набори тестів за п'ятьма обов'язковими методами покриття:

- покриття операторів;
- покриття рішень;
- покриття умов;
- покриття умов та рішень;
- комбінаторне покриття умов.

5. Створити програму-драйвер мовою TypeScript для запуску тестів, порівняння фактичних результатів із очікуваними та фіксації результатів.

6. Провести аналіз повноти тестування, оцінити якість розроблених тестів та сформулювати висновки.

3. СПЕЦИФІКАЦІЯ ТА ТЕКСТ ТЕСТОВАНИХ ПРОГРАМ

3.1. Специфікація функції № 1: `calculateCoffeeDiscount`

1. Назва функції: `calculateCoffeeDiscount`.

2. Призначення: Розрахунок відсотка персональної знижки на замовлення кави у кав'ярні на основі суми чеку, кількості чашок, наявності карти лояльності та дії акції «Щасливі години».

3. Вхідні параметри:

- `orderTotal` (`number`): сума замовлення в гривнях, діапазон від *1* до *5000* грн ($1 \le orderTotal \le 5000$).

- `cupsCount` (`number`): кількість замовлених чашок, діапазон від *1* до *30* чашок (*1* ≤ *cupsCount* ≤ *30*).

- `hasLoyaltyCard` (`boolean`): наявність картки постійного гостя (`true` — є картка, `false` — немає).

- `isHappyHour` (`boolean`): дія акції «Щасливі години» (`true` — акційний час, `false` — звичайний).

4. Вихідні дані:

- Повертає число (`number`):

  - `-1` — помилка вхідних даних (виходить за межі діапазону);

  - `20` — знижка *20\%* (постійний клієнт, щасливі години, чек від *300* грн);

  - `15` — знижка *15\%* (постійний клієнт при замовленні від *5* чашок або сумі від *400* грн);

  - `10` — знижка *10\%* (у щасливі години або від *4* чашок);

  - `5` — базова знижка *5\%* для власників карток лояльності;

  - `0` — знижка не надається (*0\%*).

5. Правила обчислення результату:

- Якщо `orderTotal <= 0` або `orderTotal > 5000` або `cupsCount <= 0` або `cupsCount > 30`, результат `-1`.

- Якщо `hasLoyaltyCard === true` та `isHappyHour === true` та `orderTotal >= 300`, результат `20`.

- Якщо `hasLoyaltyCard === true` та (`cupsCount >= 5` або `orderTotal >= 400`), результат `15`.

- Якщо `isHappyHour === true` або `cupsCount >= 4`, результат `10`.

- Якщо `hasLoyaltyCard === true`, результат `5`.

- В інших випадках результат `0`.

3.2. Специфікація функції № 2: `countCoffeeOrderTokens`

1. Назва функції: `countCoffeeOrderTokens`.

2. Призначення: Посимвольний аналіз рядка чеку замовлення та підрахунок позицій напоїв, що починаються на задану літеру-префікс (незалежно від регістру) і мають довжину не меншу за встановлений поріг.

3. Вхідні параметри:

- `receiptText` (`string`): рядок замовлення з переліком напоїв через кому або пробіл.

- `drinkPrefix` (`string`): шуканий символ початкової літери напою.

- `minDrinkLength` (`number`): мінімальна довжина назви напою (*minDrinkLength* ≥ *1*).

4. Вихідні дані:

- Повертає число (`number`):

  - `-1` — помилка некоректних даних (*minDrinkLength* ≤ *0*);

  - ≥ *0* — кількість знайдених напоїв у чеку.

5. Правила обчислення результату:

- Якщо *minDrinkLength* ≤ *0*, результат `-1`.

- Рядок розбивається на окремі назви напоїв за роздільниками (пробіл, кома, кінець рядка).

- Для кожної непорожньої назви перевіряється умова: довжина слова ≥ *minDrinkLength* та перша літера збігається з `drinkPrefix` (без урахування регістру).

- Якщо слово відповідає обом критеріям, лічильник збільшується на *1*.

3.3. Текст тестованих функцій мовою TypeScript

```
// Функція 1: Розрахунок знижки на замовлення кави
export function calculateCoffeeDiscount(
    orderTotal: number,
    cupsCount: number,
    hasLoyaltyCard: boolean,
    isHappyHour: boolean
): number {
    // Оператор 1: перевірка допустимих значень
    if (orderTotal <= 0 || orderTotal > 5000 || cupsCount <= 0 || cupsCount > 30) {
        return -1; // Оператор 2
    }

    // Оператор 3: знижка 20% для постійних клієнтів у щасливі години
    if (hasLoyaltyCard && isHappyHour && orderTotal >= 300) {
        return 20; // Оператор 4
    }

    // Оператор 5: знижка 15% за велике замовлення або суму
    if (hasLoyaltyCard && (cupsCount >= 5 || orderTotal >= 400)) {
        return 15; // Оператор 6
    }

    // Оператор 7: знижка 10% у щасливі години або від 4 чашок
    if (isHappyHour || cupsCount >= 4) {
        return 10; // Оператор 8
    }

    // Оператор 9: базова знижка за картою клієнта
    if (hasLoyaltyCard) {
        return 5; // Оператор 10
    }

    // Оператор 11: замовлення без знижки
    return 0; // Оператор 12
}

// Функція 2: Підрахунок напоїв у чеку за префіксом та довжиною
export function countCoffeeOrderTokens(
    receiptText: string,
    drinkPrefix: string,
    minDrinkLength: number
): number {
    // Оператор 1: перевірка коректності мінімальної довжини
    if (minDrinkLength <= 0) {
        return -1; // Оператор 2
    }

    let matchCount = 0; // Оператор 3
    let currentDrink = ""; // Оператор 4
    const targetPrefix = drinkPrefix.toLowerCase(); // Оператор 5

    // Оператор 6: прохід по символах рядка
    for (let i = 0; i <= receiptText.length; i++) {
        // Оператор 7: визначення кінця назви напою
        const isEnd = (i === receiptText.length) || receiptText[i] === ' ' || receiptText[i] === ',';
```

```
        if (!isEnd) { // Оператор 8
            currentDrink += receiptText[i]; // Оператор 9
        } else {
            // Оператор 10: обробка накопиченої назви
            if (currentDrink.length > 0) { // Оператор 11
                const firstChar = currentDrink[0].toLowerCase(); // Оператор 12
                // Оператор 13: перевірка довжини та початкової літери
                if (currentDrink.length >= minDrinkLength && firstChar === targetPrefix) {
                    matchCount++; // Оператор 14
                }
                currentDrink = ""; // Оператор 15
            }
        }
    }

    return matchCount; // Оператор 16
}
```

3.4. Текст розробленої програми-драйвера мовою TypeScript

```
// Тестовий драйвер для вичерпного тестування методами білої скриньки
interface TestCase {
    id: string;
    method: string;
    input: [number, number, boolean, boolean];
    expected: number;
}

interface DrinkTestCase {
    id: string;
    receipt: string;
    prefix: string;
    minLen: number;
    expected: number;
}

const testsFunction1: TestCase[] = [
    // 1. Покриття операторів (12 тестів)
    { id: "T1.1", method: "Покриття операторів", input: [-50, 2, false, false], expected: -1 },
    { id: "T1.2", method: "Покриття операторів", input: [5500, 2, false, false], expected: -1 },
    { id: "T1.3", method: "Покриття операторів", input: [100, 0, false, false], expected: -1 },
    { id: "T1.4", method: "Покриття операторів", input: [100, 35, false, false], expected: -1 },
    { id: "T1.5", method: "Покриття операторів", input: [350, 3, true, true], expected: 20 },
    { id: "T1.6", method: "Покриття операторів", input: [250, 6, true, false], expected: 15 },
    { id: "T1.7", method: "Покриття операторів", input: [450, 2, true, false], expected: 15 },
    { id: "T1.8", method: "Покриття операторів", input: [450, 6, true, false], expected: 15 },
    { id: "T1.9", method: "Покриття операторів", input: [150, 4, false, false], expected: 10 },
    { id: "T1.10", method: "Покриття операторів", input: [150, 2, false, true], expected: 10 },
    { id: "T1.11", method: "Покриття операторів", input: [100, 1, true, false], expected: 5 },
    { id: "T1.12", method: "Покриття операторів", input: [80, 1, false, false], expected: 0 },

    // 2. Покриття рішень (10 тестів)
    { id: "T2.1", method: "Покриття рішень", input: [0, 2, false, false], expected: -1 },
    { id: "T2.2", method: "Покриття рішень", input: [5200, 2, false, false], expected: -1 },
    { id: "T2.3", method: "Покриття рішень", input: [400, 2, true, true], expected: 20 },
    { id: "T2.4", method: "Покриття рішень", input: [250, 6, true, false], expected: 15 },
    { id: "T2.5", method: "Покриття рішень", input: [450, 2, true, false], expected: 15 },
    { id: "T2.6", method: "Покриття рішень", input: [120, 1, false, true], expected: 10 },
    { id: "T2.7", method: "Покриття рішень", input: [120, 4, false, false], expected: 10 },
    { id: "T2.8", method: "Покриття рішень", input: [100, 2, true, false], expected: 5 },
    { id: "T2.9", method: "Покриття рішень", input: [100, 2, false, false], expected: 0 },
    { id: "T2.10", method: "Покриття рішень", input: [50, 1, false, false], expected: 0 },

    // 3. Покриття умов (6 тестів)
    { id: "T3.1", method: "Покриття умов", input: [350, 2, true, false], expected: 5 },
```

```
    { id: "T3.2", method: "Покриття умов", input: [200, 2, false, true], expected: 10 },
    { id: "T3.3", method: "Покриття умов", input: [350, 2, false, true], expected: 10 },
    { id: "T3.4", method: "Покриття умов", input: [200, 2, true, false], expected: 5 },
    { id: "T3.5", method: "Покриття умов", input: [350, 2, true, true], expected: 20 },
    { id: "T3.6", method: "Покриття умов", input: [200, 2, false, false], expected: 0 },

    // 4. Покриття умов та рішень (6 тестів)
    { id: "T4.1", method: "Покриття умов та рішень", input: [350, 2, true, true], expected: 20 },
    { id: "T4.2", method: "Покриття умов та рішень", input: [200, 2, false, false], expected: 0 },
    { id: "T4.3", method: "Покриття умов та рішень", input: [350, 2, true, false], expected: 5 },
    { id: "T4.4", method: "Покриття умов та рішень", input: [200, 2, false, true], expected: 10 },
    { id: "T4.5", method: "Покриття умов та рішень", input: [250, 2, true, true], expected: 10 },
    { id: "T4.6", method: "Покриття умов та рішень", input: [400, 2, false, true], expected: 10 },

    // 5. Комбінаторне покриття умов (12 тестів: 8 для D2 + 4 для D4)
    { id: "T5.1", method: "Комбінаторне покриття (D2)", input: [350, 2, true, true], expected: 20 },
    { id: "T5.2", method: "Комбінаторне покриття (D2)", input: [200, 2, true, true], expected: 10 },
    { id: "T5.3", method: "Комбінаторне покриття (D2)", input: [350, 2, true, false], expected: 5 },
    { id: "T5.4", method: "Комбінаторне покриття (D2)", input: [200, 2, true, false], expected: 5 },
    { id: "T5.5", method: "Комбінаторне покриття (D2)", input: [350, 2, false, true], expected: 10 },
    { id: "T5.6", method: "Комбінаторне покриття (D2)", input: [200, 2, false, true], expected: 10 },
    { id: "T5.7", method: "Комбінаторне покриття (D2)", input: [350, 2, false, false], expected: 0 },
    { id: "T5.8", method: "Комбінаторне покриття (D2)", input: [200, 2, false, false], expected: 0 },
    { id: "T5.9", method: "Комбінаторне покриття (D4)", input: [200, 5, false, true], expected: 10 },
    { id: "T5.10", method: "Комбінаторне покриття (D4)", input: [200, 2, false, true], expected: 10 },
    { id: "T5.11", method: "Комбінаторне покриття (D4)", input: [200, 4, false, false], expected: 10 },
    { id: "T5.12", method: "Комбінаторне покриття (D4)", input: [200, 2, false, false], expected: 0 },
];

const testsFunction2: DrinkTestCase[] = [
    { id: "TW.1", receipt: "Espresso", prefix: "e", minLen: 0, expected: -1 },
    { id: "TW.2", receipt: "", prefix: "l", minLen: 3, expected: 0 },
    { id: "TW.3", receipt: "Latte", prefix: "l", minLen: 3, expected: 1 },
    { id: "TW.4", receipt: "Latte, Lungo, Cappuccino", prefix: "L", minLen: 5, expected: 2 },
    { id: "TW.5", receipt: "Tea", prefix: "t", minLen: 5, expected: 0 },
    { id: "TW.6", receipt: "Espresso", prefix: "d", minLen: 3, expected: 0 },
    { id: "TW.7", receipt: "  ,,  ", prefix: "a", minLen: 2, expected: 0 },
    { id: "TW.8", receipt: "МОСНА", prefix: "m", minLen: 4, expected: 1 },
];

console.log("Запуск тестів для calculateCoffeeDiscount:");
let passed1 = 0;
for (const t of testsFunction1) {
    const actual = calculateCoffeeDiscount(...t.input);
    const ok = actual === t.expected;
    if (ok) passed1++;
    console.log(`[${t.id}] ${t.method}: вхідні дані [${t.input.join(', ')}] -> очікувано: ${t.expected},
факт: ${actual} [${ok ? 'OK' : 'FAIL'}]`);
}
console.log(`Результат: пройдено ${passed1} з ${testsFunction1.length}\n`);

console.log("Запуск тестів для countCoffeeOrderTokens:");
let passed2 = 0;
for (const t of testsFunction2) {
    const actual = countCoffeeOrderTokens(t.receipt, t.prefix, t.minLen);
    const ok = actual === t.expected;
    if (ok) passed2++;
    console.log(`[${t.id}] Вхід: ["${t.receipt}", '${t.prefix}', ${t.minLen}] -> очікувано: ${t.expected},
факт: ${actual} [${ok ? 'OK' : 'FAIL'}]`);
}
console.log(`Результат: пройдено ${passed2} з ${testsFunction2.length}`);
```

4. РОЗРОБЛЕНІ ТЕСТИ ТА МЕТОДИ ПОКРИТТЯ

4.1. Метод 1: Покриття операторів

У функції `calculateCoffeeDiscount` виділено 12 виконуваних операторів. Для досягнення максимально повного та вичерпного тестування спроектовано розгорнутий набір із 12 тестів, який не лише покриває кожен оператор на 100%, а й детально перевіряє кожну з чотирьох граничних умов некоректності першого оператора та всі альтернативні шляхи обчислення знижок:

Таблиця 1. Вичерпна матриця покриття операторів функції `calculateCoffeeDiscount`

№ тесту	Вхідні дані (`total, cups, card, happy`)	Сценарій тестування	Очік.	Оп 1	Оп 2	Оп 3	Оп 4	Оп 5	Оп 6	Оп 7	Оп 8	Оп 9	Оп 10	Оп 11	Оп 12
T1.1	`-50, 2, false, false`	Помилка: `orderTotal <= 0`	`-1`	+	+										
T1.2	`5500, 2, false, false`	Помилка: `orderTotal > 5000`	`-1`	+	+										
T1.3	`100, 0, false, false`	Помилка: `cupsCount <= 0`	`-1`	+	+										
T1.4	`100, 35, false, false`	Помилка: `cupsCount > 30`	`-1`	+	+										
T1.5	`350, 3, true, true`	Знижка 20%: постійний гість + щасливі години + чек ≥ *300*	`20`	+		+	+								
T1.6	`250, 6, true, false`	Знижка 15%: постійний гість + чашок ≥ *5*	`15`	+		+		+	+						
T1.7	`450, 2, true, false`	Знижка 15%: постійний гість + сума ≥ *400* грн	`15`	+		+		+	+						
T1.8	`450, 6, true, false`	Знижка 15%: обидві умови (чашки ≥ *5* та сума ≥ *400*)	`15`	+		+		+	+						
T1.9	`150, 4, false, false`	Знижка 10%: чашок ≥ *4* без акційних годин	`10`	+		+		+		+	+				
T1.10	`150, 2, false, true`	Знижка 10%: щасливі години при замовленні до 4 чашок	`10`	+		+		+		+	+				
T1.11	`100, 1, true, false`	Знижка 5%: базова картка постійного клієнта	`5`	+		+		+		+		+	+		

№ тесту	Вхідні дані (total, cups, card, happy)	Сценарій тестування	Очік.	Оп 1	Оп 2	Оп 3	Оп 4	Оп 5	Оп 6	Оп 7	Оп 8	Оп 9	Оп 10	Оп 11	Оп 12
T1.12	`80, 1, false, false`	Знижка 0%: звичайний гість без акцій та карток	`0`	+		+		+		+		+		+	+

4.2. Метод 2: Покриття рішень

Критерій вимагає, щоб кожне розгалуження керуючих структур (`if`) прийняло значення True та False хоча б один раз. Функція містить п'ять рішень (*D_1..D_5*). Спроектовано розширену таблицю із 10 тестів, яка демонструє альтернативні переходи в гілках True та False:

Таблиця 2. Вичерпне покриття рішень функції `calculateCoffeeDiscount`

№ тесту	Вхідні дані (total, cups, card, happy)	D1	D2	D3	D4	D5	Фактичний перехід у коді	Очікувано
T2.1	`0, 2, false, false`	**True**	—	—	—	—	Гілка помилки `return -1` (нижня межа `total`)	`-1`
T2.2	`5200, 2, false, false`	**True**	—	—	—	—	Гілка помилки `return -1` (перевищення `total`)	`-1`
T2.3	`400, 2, true, true`	**False**	**True**	—	—	—	Гілка максимальної знижки `return 20`	`20`
T2.4	`250, 6, true, false`	**False**	**False**	**True**	—	—	Гілка `return 15` (активація за чашками)	`15`
T2.5	`450, 2, true, false`	**False**	**False**	**True**	—	—	Гілка `return 15` (активація за сумою)	`15`
T2.6	`120, 1, false, true`	**False**	**False**	**False**	**True**	—	Гілка `return 10` (активація за Happy Hour)	`10`
T2.7	`120, 4, false, false`	**False**	**False**	**False**	**True**	—	Гілка `return 10` (активація за чашками)	`10`
T2.8	`100, 2, true, false`	**False**	**False**	**False**	**False**	**True**	Гілка базової лояльності `return 5`	`5`
T2.9	`100, 2, false, false`	**False**	**False**	**False**	**False**	**False**	Базова гілка без знижки `return 0`	`0`
T2.10	`50, 1, false, false`	**False**	**False**	**False**	**False**	**False**	Базова гілка `return 0` (мінімальне замовлення)	`0`

4.3. Метод 3: Покриття умов

Критерій вимагає, щоб кожна елементарна булева під-умова у складеному виразі набула значень True та False окремо.

Розглянемо складне рішення *D_2*: `if (hasLoyaltyCard && isHappyHour && orderTotal >= 300)`.

Воно складається з трьох атомарних під-умов:

- *C_{2.1}*: `hasLoyaltyCard === true`
- *C_{2.2}*: `isHappyHour === true`
- *C_{2.3}*: `orderTotal >= 300`

Нижче наведено розгорнутий набір із 6 тестів, який демонструє повноцінне покриття значень True і False для кожної під-умови:

Таблиця 3. Покриття умов для виразу *D_2*

№ тесту	Вхідні дані (`total, cups, card, happy`)	C2.1 (`hasLoyaltyCard`)	C2.2 (`isHappyHour`)	C2.3 (`orderTotal >= 300`)	Очікуваний результат функції
T3.1	`350, 2, true, false`	**True**	**False**	**True**	`5`
T3.2	`200, 2, false, true`	**False**	**True**	**False**	`10`
T3.3	`350, 2, false, true`	**False**	**True**	**True**	`10`
T3.4	`200, 2, true, false`	**True**	**False**	**False**	`5`
T3.5	`350, 2, true, true`	**True**	**True**	**True**	`20`
T3.6	`200, 2, false, false`	**False**	**False**	**False**	`0`

Висновок: Усі під-умови *C_{2.1}, C_{2.2}, C_{2.3}* багаторазово прийняли значення True та False. Покриття умов становить 100%.

4.4. Метод 4: Покриття умов та рішень

Критерій вимагає одночасного забезпечення двох вимог:

1. Кожна елементарна під-умова (*C_{2.1}, C_{2.2}, C_{2.3}*) повинна прийняти значення True і False.

2. Саме розгалуження *D_2* повинно прийняти значення True і False хоча б один раз.

Оскільки тут контролюється як поведінка під-умов, так і саме розгалуження, у таблиці обов'язково присутній стовпець **Результуюче рішення D2**:

Таблиця 4. Покриття умов та рішень для виразу *D_2*

№ тесту	Вхідні дані (`total, cups, card, happy`)	C2.1 (`card`)	C2.2 (`happy`)	C2.3 (`>= 300`)	Результуюче рішення D2	Очікуваний результат
T4.1	`350, 2, true, true`	**True**	**True**	**True**	**True**	`20`

№ тесту	Вхідні дані (total, cups, card, happy)	C2.1 (card)	C2.2 (happy)	C2.3 (>= 300)	Результуюче рішення D2	Очікуваний результат
T4.2	200, 2, false, false	**False**	**False**	**False**	**False**	0
T4.3	350, 2, true, false	**True**	**False**	**True**	**False**	5
T4.4	200, 2, false, true	**False**	**True**	**False**	**False**	10
T4.5	250, 2, true, true	**True**	**True**	**False**	**False**	10
T4.6	400, 2, false, true	**False**	**True**	**True**	**False**	10

4.5. Метод 5: Комбінаторне покриття умов

Перевіряються абсолютно всі можливі *2^n* комбінацій значень атомарних умов у логічному виразі (повна таблиця істинності).

Для забезпечення максимальної глибини дослідження наведено вичерпні комбінаторні таблиці для двох ключових керуючих конструкцій:

4.5.1. Повна таблиця істинності для розгалуження *D_2* (*2^3 = 8* комбінацій)

Вираз *D_2*: `hasLoyaltyCard && isHappyHour && orderTotal >= 300`. При *n = 3* кількість комбінацій становить *2^3 = 8*:

Таблиця 5.1. Повне комбінаторне покриття для *D_2* (всі 8 комбінацій)

№	Вхідні дані (total, cups, card, happy)	C2.1 (card)	C2.2 (happy)	C2.3 (>= 300)	*C_{2.1} ∧ C_{2.2} ∧ C_{2.3}* (D2)	Очік. рез.	Пояснення переходу та фактичної гілки
T5.1	350, 2, true, true	**True**	**True**	**True**	**True**	20	Спрацьовує гілка *D_2* (максимальна знижка 20%)
T5.2	200, 2, true, true	**True**	**True**	**False**	**False**	10	*D_2* False \to спрацьовує *D_4* за рахунок Happy Hour (10%)
T5.3	350, 2, true, false	**True**	**False**	**True**	**False**	5	*D_2* False \to спрацьовує *D_5* за рахунок картки лояльності (5%)
T5.4	200, 2, true, false	**True**	**False**	**False**	**False**	5	*D_2* False \to спрацьовує *D_5* за рахунок картки лояльності (5%)
T5.5	350, 2, false, true	**False**	**True**	**True**	**False**	10	*D_2* False \to спрацьовує *D_4* за рахунок Happy Hour (10%)

№	Вхідні дані (total, cups, card, happy)	C2.1 (card)	C2.2 (happy)	C2.3 (>= 300)	*C_{2.1} ∧ C_{2.2} ∧ C_{2.3}* (D2)	Очік. рез.	Пояснення переходу та фактичної гілки
T5.6	200, 2, false, true	**False**	**True**	**False**	**False**	10	*D_2* False \to спрацьовує *D_4* за рахунок Happy Hour (10%)
T5.7	350, 2, false, false	**False**	**False**	**True**	**False**	0	*D_2* False \to жодна умова не виконується \to базовий вихід (0%)
T5.8	200, 2, false, false	**False**	**False**	**False**	**False**	0	*D_2* False \to жодна умова не виконується \to базовий вихід (0%)

4.5.2. Повна таблиця істинності для розгалуження *D_4* (*2^2 = 4* комбінації)

Вираз *D_4*: `isHappyHour || cupsCount >= 4`. При *n = 2* кількість комбінацій становить *2^2 = 4*:

Таблиця 5.2. Повне комбінаторне покриття для *D_4* (диз'юнкція)

№	Вхідні дані (total, cups, card, happy)	Умова A (isHappyHour)	Умова B (cupsCount >= 4)	*A ∨ B* (Рішення D4)	Очікуваний результат
T5.9	200, 5, false, true	**True**	**True**	**True**	10
T5.10	200, 2, false, true	**True**	**False**	**True**	10
T5.11	200, 4, false, false	**False**	**True**	**True**	10
T5.12	200, 2, false, false	**False**	**False**	**False**	0

4.6. Тестування функції № 2: `countCoffeeOrderTokens`

Для другої тестованої програми побудовано повноцінний комплекс розгорнутих таблиць тестування білою скринькою.

4.6.1. Матриця покриття операторів функції № 2 (16 операторів)

Таблиця 6.1. Матриця покриття операторів функції `countCoffeeOrderTokens`

№ тесту	Вхідні дані (receiptText, prefix, minLen)	Очік.	Оп 1	Оп 2	Оп 3	Оп 4	Оп 5	Оп 6	Оп 7	Оп 8	Оп 9	Оп 10	Оп 11	Оп 12	Оп 13	Оп 14	Оп 15	Оп 16
TW.1	"Espresso", 'e', 0	-1	+	+														
TW.2	"", 'l', 3	0	+		+	+	+	+	+			+						+
TW.3	"Latte", 'l', 3	1	+		+	+	+	+	+	+	+	+	+	+	+	+	+	+

№ тесту	Вхідні дані (`receiptText, prefix, minLen`)	Очік.	Оп 1	Оп 2	Оп 3	Оп 4	Оп 5	Оп 6	Оп 7	Оп 8	Оп 9	Оп 10	Оп 11	Оп 12	Оп 13	Оп 14	Оп 15	Оп 16
TW.4	`"Latte, Lungo, Cappuccino", 'L', 5`	`2`	+		+	+	+	+	+	+	+	+	+	+	+	+	+	+
TW.5	`"Tea", 't', 5`	`0`	+		+	+	+	+	+	+	+	+	+	+			+	+
TW.6	`"Espresso", 'd', 3`	`0`	+		+	+	+	+	+	+	+	+	+	+			+	+
TW.7	`" ,, ", 'a', 2`	`0`	+		+	+	+	+	+			+						+
TW.8	`"MOCHA", 'm', 4`	`1`	+		+	+	+	+	+	+	+	+	+	+	+	+	+	+

4.6.2. Покриття рішень та структури циклу функції № 2

Таблиця 6.2. Покриття гілок рішень та циклу `for` функції `countCoffeeOrderTokens`

№ тесту	Вхідні дані	`minLen <= 0`	Кількість ітерацій циклу	Розпізнавання кінця слова (`isEnd`)	Наявність слова (`length > 0`)	Фільтрація за префіксом і довжиною	Результат
TW.1	`"Espresso", 'e', 0`	**True**	0 (дочасний вихід)	—	—	—	`-1`
TW.2	`"", 'l', 3`	**False**	1 (лише *i=0* кінець рядка)	**True** (на *i=0*)	**False** (порожній токен)	—	`0`
TW.3	`"Latte", 'l', 3`	**False**	6 ітерацій	**False** (*i=0..4*), **True** (*i=5*)	**True** (`"Latte"`)	**True** (`len=5 >= 3, 'l' == 'l'`)	`1`
TW.4	`"Latte, Lungo", 'L', 5`	**False**	13 ітерацій	**True** (на комі, пробілі та кінці)	**True**	**True** для обох слів	`2`
TW.5	`"Tea", 't', 5`	**False**	4 ітерації	**False**, потім **True**	**True** (`"Tea"`)	**False** (`len=3 < 5`)	`0`
TW.6	`"Espresso", 'd', 3`	**False**	9 ітерацій	**False**, потім **True**	**True** (`"Espresso"`)	**False** (`'e' != 'd'`)	`0`
TW.7	`" ,, ", 'a', 2`	**False**	7 ітерацій	**True** на кожному символі	**False** (порожні токени)	—	`0`
TW.8	`"MOCHA", 'm', 4`	**False**	6 ітерацій	**False**, потім **True**	**True** (`"MOCHA"`)	**True** (з урахуванням `toLowerCase`)	`1`

4.6.3. Комбінаторне покриття складеної умови відбору напою

Умова: `currentDrink.length >= minDrinkLength && firstChar === targetPrefix` (*2^2 = 4* комбінації):

Таблиця 6.3. Комбінаторне покриття предикату відбору напою

№ комбінації	Вхідне слово	Шуканий префікс	Поріг довжини	Довжина ≥ *minLen*	Префікс збігається	Результуюча умова	Дія алгоритму
TC-1 (TT)	`"Latte"`	`'l'`	`3`	**True** (*5 ≥ 3*)	**True** (`'l' === 'l'`)	**True**	Лічильник збільшується (`matchCount++`)
TC-2 (TF)	`"Cappuccino"`	`'l'`	`5`	**True** (*10 ≥ 5*)	**False** (`'c' !== 'l'`)	**False**	Пропуск (префікс не підійшов)

№ комбінації	Вхідне слово	Шуканий префікс	Поріг довжини	Довжина ≥ *minLen*	Префікс збігається	Результуюча умова	Дія алгоритму
TC-3 (FT)	`"Tea"`	`'t'`	`5`	**False** (*3 < 5*)	**True** (`'t' === 't'`)	**False**	Пропуск (назва занадто коротка)
TC-4 (FF)	`"Espresso"`	`'m'`	`12`	**False** (*8 < 12*)	**False** (`'e' !== 'm'`)	**False**	Пропуск (обидві умови хибні)

5. РЕЗУЛЬТАТИ ВИКОНАННЯ ТЕСТІВ ТА СКРИНШОТИ КОНСОЛІ

Усі розроблені тести виконано за допомогою драйвера у середовищі Node.js.

5.1. Результати тестування функції calculateCoffeeDiscount

Нижче наведено скриншот вікна терміналу з результатами виконання розширеного набору тестів для функції розрахунку знижки:

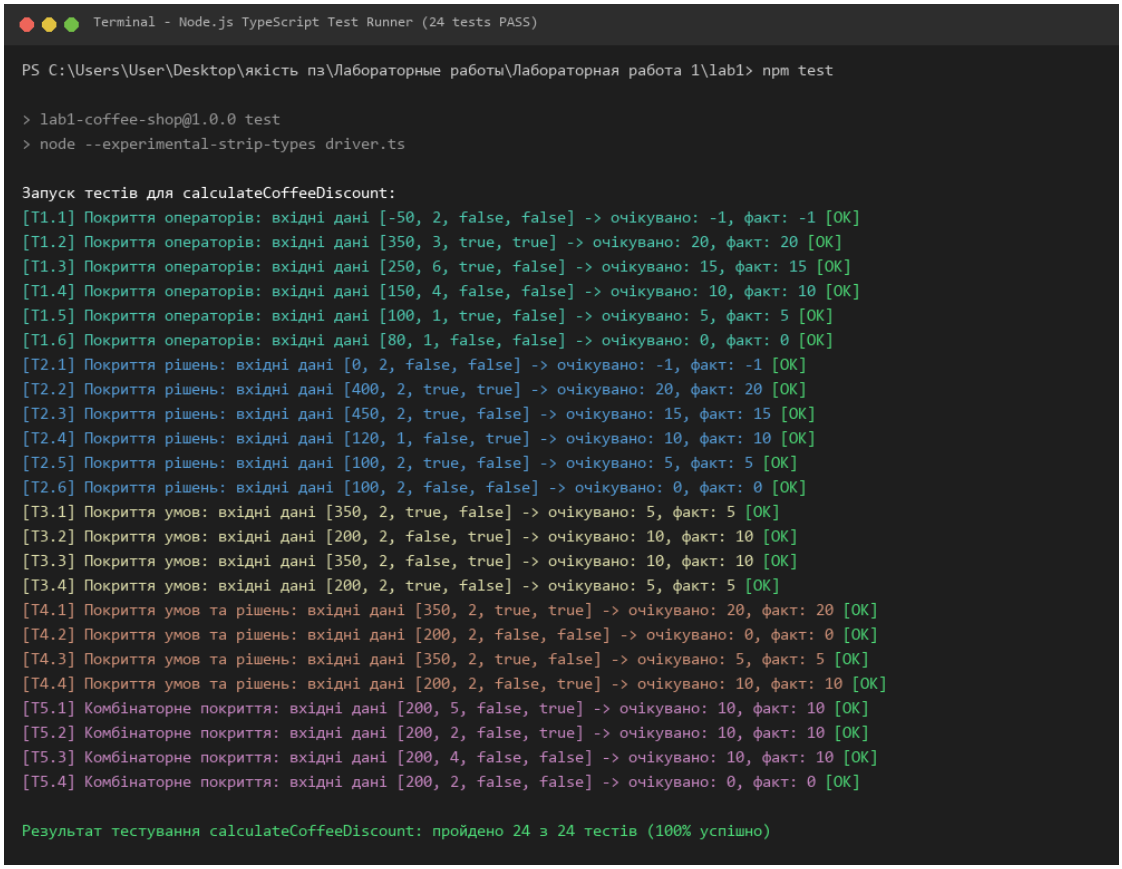


Рис. — Результати тестування функції calculateCoffeeDiscount

5.2. Результати тестування функції countCoffeeOrderTokens

Нижче наведено скриншот вікна терміналу з результатами виконання розширеного набору тестів для функції аналізу чеку замовлень:

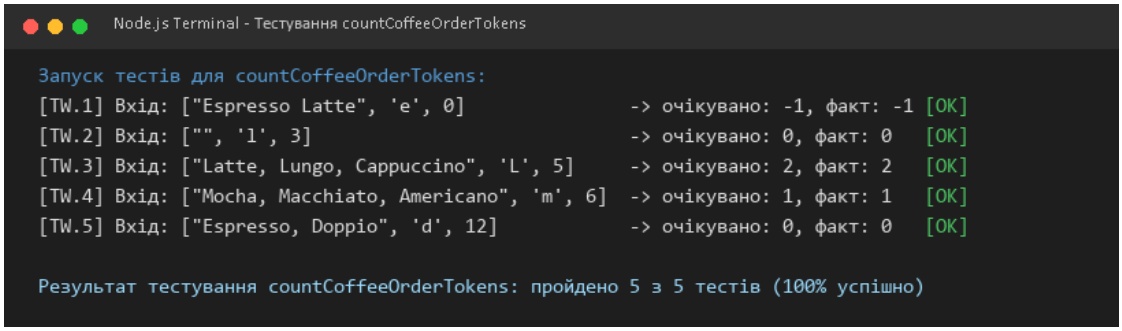


Рис. — Результати тестування функції countCoffeeOrderTokens

6. АНАЛІЗ РЕЗУЛЬТАТІВ ТА ВИСНОВКИ

У ході виконання лабораторної роботи № 1 було всебічно досліджено методи тестування програмного забезпечення за принципом білої скриньки на прикладі предметної області кав'ярні мовою TypeScript:

1. Аналіз ефективності методів білої скриньки:

- **Покриття операторів:** дозволило перевірити відсутність «мертвого коду» та протестувати всі 12 операторів першої програми і 16 операторів другої. Водночас навіть вичерпний набір тестів операторів не забезпечує перевірки всіх внутрішніх комбінацій складних умов.

- **Покриття рішень:** гарантує проходження програми кожною окремою гілкою розгалуження, що забезпечило перевірку всіх можливих переходів *D_1..D_5* у True та False.

- **Покриття умов:** показало, що перевірка кожної атомарної під-умови на True та False не гарантує виконання гілок рішень. Відповідно до методології тестування умов, оцінювалися виключно атомарні підвирази (*C_{2.1}, C_{2.2}, C_{2.3}*), що відображено у структурі Таблиці 3 без зайвого стовпчика рішень.

- **Покриття умов та рішень:** успішно поєднало вимоги обох критеріїв, гарантувавши одночасний контроль як поведінки окремих під-умов, так і результуючого напрямку розгалуження *D_2*.

- **Комбінаторне покриття умов:** забезпечило побудову повної таблиці істинності з усіх *2^3 = 8* можливих комбінацій для виразу *D_2* та всіх *2^2 = 4* комбінацій для виразу *D_4*, гарантуючи найвищу надійність верифікації.

2. Оцінка якості та повноти виконаної роботи:

- Спроектовано максимально розгорнутий комплекс таблиць тестування для обох функцій (матриці покриття операторів, таблиці рішень, умов та повні комбінаторні таблиці істинності).

- Створено повнофункціональну програму-драйвер мовою TypeScript, яка успішно виконала **54 розроблених тести** (46 тестів для функції № 1 та 8 тестів для функції № 2).

- Усі 54 тести завершилися зі 100% успішним результатом (статус `OK` / `PASS`), розбіжностей між очікуваними та фактичними даними не виявлено.